SOFTWARE ENGINEERING LABORATORY

Lab 07: Fuzz Testing

Weather Alert Classifier

Ved Mudkavi | PES1UG24CS522
12 November 2026

Objective

Use Hypothesis to uncover input-validation failures in all three assigned weather functions, fix the application, and rerun the same tests.

Before fixing	After fixing	Evidence
6 failed, 10 passed (0.92 s)	16 passed (0.91 s)	Six original terminal screenshots

Requirements and evidence

Handout requirement	Completion / location
Python 3.8+, Hypothesis and pytest	Python 3.12.14, Hypothesis 6.170.15, pytest 9.2.0; visible in test screenshots.
Step 2: run the original application	Screenshot 1, page 2.
Step 3: complete tests for all three functions	test_weather_alert.py; property tests and regression checks.
Step 4: show failing tests	Screenshots 2-4, pages 3-5.
Step 5: fix and run the application	weather_alert.py; screenshot 5, page 6.
Step 6: rerun the same tests	Screenshot 6, page 7: all 16 cases pass.
Deliverables	This screenshot PDF, fixed weather_alert.py, and completed test_weather_alert.py.

Validation decisions

Invalid inputs return **None**; the interactive program displays a clear message and continues. Reading temperature must be finite and humidity an integer from 0 to 100. Wind speed must be a nonnegative integer. Rainfall must be a nonempty CSV of finite, nonnegative values. Valid outputs and the original wind thresholds (39 and 89 km/h) are preserved.

Source: Lab 7 Fuzz Testing Student Handout, steps 1-6. Assigned starter: SETAPESU26/L6_L7_32_weather_alert / lab7. The handout's generic processor filenames correspond to weather_alert.py and test_weather_alert.py in this assignment.

Original application output

Step 2 | Screenshot 1 | Captured at 11:10:21 AM

```
Last login: Sun Nov  1 07:59:06 on ttys000
(base) vedmudkavi@Veds-MacBook-Air-2 ~ % LAB="/Users/vedmudkavi/Documents/Codex/2026-11-12/la"
source "$LAB/work/lab7/venv/bin/activate"
((venv) ) (base) vedmudkavi@Veds-MacBook-Air-2 ~ % cd "$LAB/work/lab7/original"
python weather_alert.py
==== Running parse_reading ====
Enter temperature/humidity (e.g. 35.5/80): 35.5/80
Temperature and Humidity: (35.5, 80)

==== Running wind_category ====
Enter a wind speed in km/h (e.g. 45): 45
Wind Category: advisory

==== Running rainfall_spread ====
Enter hourly rainfall in mm as CSV (e.g. 2,5.5,10): 2,5.5,10
Rainfall Spread: 8.0
((venv) ) (base) vedmudkavi@Veds-MacBook-Air-2 original %
```

The assigned application is run before fixing. Input 35.5/80 gives (35.5, 80), wind speed 45 gives advisory, and rainfall 2,5.5,10 gives a spread of 8.0. This establishes the original valid behavior.

Fuzz testing before the fix

Steps 3-4 | Screenshot 2 | Captured at 11:10:44 AM

```
[((venv) ) (base) vedmudkavi@Veds-MacBook-Air-2 original % python -m pytest test_weather_alert.py -v --tb=short --hypothesis-seed=732                                                
```

Hypothesis failure details

Step 4 | Screenshot 3 | Captured at 11:11:00 AM

```
=================================== FAILURES ===================================
_____ test_parse_reading_no_crash _____
test_weather_alert.py:27: in test_parse_reading_no_crash
    def test_parse_reading_no_crash(value):
                   ^^^
test_weather_alert.py:28: in test_parse_reading_no_crash
    result = parse_reading(value)
             ^^^^^^^^^^^^^^^^^^^^
weather_alert.py:2: in parse_reading
    temp, humidity = text.split('/')
                     ^^^^^^^^^^
E   AttributeError: 'NoneType' object has no attribute 'split'
E   Failing test case: test_parse_reading_no_crash(
E       value=None,
E   )
_____ test_wind_category_no_crash _____
test_weather_alert.py:37: in test_wind_category_no_crash
    def test_wind_category_no_crash(value):
                   ^^^
test_weather_alert.py:38: in test_wind_category_no_crash
    assert wind_category(value) in (None, "none", "advisory", "warning")
           ^^^^^^^^^^^^^^^^^^^^
weather_alert.py:6: in wind_category
    speed = int(speed_text)
            ^^^^^^^^^^^^^^^
E   ValueError: invalid literal for int() with base 10: ''
E   Failing test case: test_wind_category_no_crash(
E       value='',
E   )
_____ test_rainfall_spread_no_crash _____
test_weather_alert.py:42: in test_rainfall_spread_no_crash
    def test_rainfall_spread_no_crash(value):
                   ^^^
test_weather_alert.py:43: in test_rainfall_spread_no_crash
    result = rainfall_spread(value)
             ^^^^^^^^^^^^^^^^^^^^^^
weather_alert.py:14: in rainfall_spread
    readings = [float(r) for r in readings_csv.split(',')]
                                  ^^^^^^^^^^^^^^^^^^^
E   AttributeError: 'NoneType' object has no attribute 'split'
E   Failing test case: test_rainfall_spread_no_crash(
E       value=None,
E   )
_____ test_parse_reading_rejects_invalid _____
test_weather_alert.py:79: in test_parse_reading_rejects_invalid
    None, "", "35.5", "35/80/90", "/80", "35/", "hot/wet",
          ^^^
test_weather_alert.py:84: in test_parse_reading_rejects_invalid
    assert parse_reading(value) is None
           ^^^^^^^^^^^^^^^^^^^^
weather_alert.py:2: in parse_reading
    temp, humidity = text.split('/')
                     ^^^^^^^^^^
E   AttributeError: 'NoneType' object has no attribute 'split'
E   Failing test case: test_parse_reading_rejects_invalid(
E       value=None,  # or any other generated value
E   )
_____ test_wind_category_rejects_invalid _____
test_weather_alert.py:88: in test_wind_category_rejects_invalid
    None, "", "fast", "-1", "45.5", "nan", "inf", "1e309",
```

Hypothesis exposes small failing inputs: None causes AttributeError in parse_reading and rainfall_spread; an empty string causes ValueError in wind_category. These failures identify missing type checks and unsafe numeric conversion.

Original test failure summary

Step 4 | Screenshot 4 | Captured at 11:11:08 AM

```
_____ test_parse_reading_rejects_invalid _____
test_weather_alert.py:79: in test_parse_reading_rejects_invalid
    None, "", "35.5", "35/80/90", "/80", "35/", "hot/wet",
               ^^^
test_weather_alert.py:84: in test_parse_reading_rejects_invalid
    assert parse_reading(value) is None
           ^^^^^^^^^^^^^^^^^^^^
weather_alert.py:2: in parse_reading
    temp, humidity = text.split('/')
                     ^^^^^^^^^^
E   AttributeError: 'NoneType' object has no attribute 'split'
E   Failing test case: test_parse_reading_rejects_invalid(
E       value=None,  # or any other generated value
E   )
_____ test_wind_category_rejects_invalid _____
test_weather_alert.py:88: in test_wind_category_rejects_invalid
    None, "", "fast", "-1", "45.5", "nan", "inf", "1e309",
               ^^^
test_weather_alert.py:92: in test_wind_category_rejects_invalid
    assert wind_category(value) is None
           ^^^^^^^^^^^^^^^^^^^^
weather_alert.py:6: in wind_category
    speed = int(speed_text)
            ^^^^^^^^^^^^^^^
E   TypeError: int() argument must be a string, a bytes-like object or a real number, not 'NoneType'
E   Failing test case: test_wind_category_rejects_invalid(
E       value=None,  # or any other generated value
E   )
_____ test_rainfall_spread_rejects_invalid _____
test_weather_alert.py:96: in test_rainfall_spread_rejects_invalid
    None, "", " ", ",", "2,,5", "2,", ",2", "wet", "-1,2",
               ^^^
test_weather_alert.py:100: in test_rainfall_spread_rejects_invalid
    assert rainfall_spread(value) is None
           ^^^^^^^^^^^^^^^^^^^^^^
weather_alert.py:14: in rainfall_spread
    readings = [float(r) for r in readings_csv.split(',')]
                                  ^^^^^^^^^^^^^^^^^^
E   AttributeError: 'NoneType' object has no attribute 'split'
E   Failing test case: test_rainfall_spread_rejects_invalid(
E       value=None,  # or any other generated value
E   )
=========== short test summary info ===========
FAILED test_weather_alert.py::test_parse_reading_no_crash - AttributeError: 'NoneType' object has no attribute 'split'
FAILED test_weather_alert.py::test_wind_category_no_crash - ValueError: invalid literal for int() with base 10: ''
FAILED test_weather_alert.py::test_rainfall_spread_no_crash - AttributeError: 'NoneType' object has no attribute 'split'
FAILED test_weather_alert.py::test_parse_reading_rejects_invalid - AttributeError: 'NoneType' object has no attribute 'split'
FAILED test_weather_alert.py::test_wind_category_rejects_invalid - TypeError: int() argument must be a string, a bytes-like object or a real number, not 'NoneType'
FAILED test_weather_alert.py::test_rainfall_spread_rejects_invalid - AttributeError: 'NoneType' object has no attribute 'split'
=========== 6 failed, 10 passed in 0.92s ===========
((venv) ) (base) vedmudkavi@Veds-MacBook-Air-2 original %
```

The remaining invalid-input failures and final result are visible: 6 failed, 10 passed in 0.92 seconds. This is the before-fix result, using the same test file later run against the fixed application.

Fixed application handles invalid inputs

Step 5 | Screenshot 5 | Captured at 11:11:48 AM

```
==================================================================================== 6 failed, 10 passed in 0.92s ====================================
((venv) ) (base) vedmudkavi@Veds-MacBook-Air-2 original % cd "$LAB/outputs/Lab-07"
python weather_alert.py
==== Running parse_reading ====
Enter temperature/humidity (e.g. 35.5/80): bad
Invalid reading: use a finite temperature / integer humidity (0–100).

==== Running wind_category ====
Enter a wind speed in km/h (e.g. 45): -1
Invalid wind speed: enter a nonnegative integer.

==== Running rainfall_spread ====
Enter hourly rainfall in mm as CSV (e.g. 2,5.5,10): 2,,5
Invalid rainfall: enter finite, nonnegative numbers separated by commas.
((venv) ) (base) vedmudkavi@Veds-MacBook-Air-2 Lab-07 %
```

The fixed application rejects bad as a weather reading, -1 as a wind speed, and 2,,5 as rainfall CSV. It displays a helpful message for each field, reaches all three prompts and returns to the terminal without an exception. The handout requires a screenshot of the fixed application; this captures that requirement.

All tests pass after the fix

Step 6 | Screenshot 6 | Captured at 11:12:07 AM

```
==== Running rainfall_spread ====
Enter hourly rainfall in mm as CSV (e.g. 2,5.5,10): 2,,5
Invalid rainfall: enter finite, nonnegative numbers separated by commas.
((venv) ) (base) vedmudkavi@Veds-MacBook-Air-2 Lab-07 % python -m pytest test_weather_alert.py -v --hypothesis-seed=732
============================= test session starts ==============================
platform darwin -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0 -- /Users/vedmudkavi/Documents/Codex/2026-11-12/la/work/lab7/venv/bin/python
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: /Users/vedmudkavi/Documents/Codex/2026-11-12/la/outputs/Lab-07
plugins: hypothesis-6.170.15
collected 16 items

test_weather_alert.py::test_parse_reading_no_crash PASSED                [  6%]
test_weather_alert.py::test_wind_category_no_crash PASSED                [ 12%]
test_weather_alert.py::test_rainfall_spread_no_crash PASSED              [ 18%]
test_weather_alert.py::test_parse_reading_valid PASSED                   [ 25%]
test_weather_alert.py::test_wind_category_valid PASSED                   [ 31%]
test_weather_alert.py::test_rainfall_spread_valid PASSED                 [ 37%]
test_weather_alert.py::test_parse_reading_rejects_invalid PASSED         [ 43%]
test_weather_alert.py::test_wind_category_rejects_invalid PASSED         [ 50%]
test_weather_alert.py::test_rainfall_spread_rejects_invalid PASSED       [ 56%]
test_weather_alert.py::test_wind_thresholds[0-none] PASSED               [ 62%]
test_weather_alert.py::test_wind_thresholds[38-none] PASSED              [ 68%]
test_weather_alert.py::test_wind_thresholds[39-advisory] PASSED          [ 75%]
test_weather_alert.py::test_wind_thresholds[88-advisory] PASSED          [ 81%]
test_weather_alert.py::test_wind_thresholds[89-warning] PASSED           [ 87%]
test_weather_alert.py::test_whitespace_and_negative_temperature PASSED   [ 93%]
test_weather_alert.py::test_single_rainfall_value PASSED                 [100%]

============================== 16 passed in 0.91s ==============================
((venv) ) (base) vedmudkavi@Veds-MacBook-Air-2 Lab-07 %
```

The unchanged suite passes all 16 cases in 0.91 seconds. This includes fuzz properties for all three functions, valid structured inputs, malformed-input rejection, wind thresholds, whitespace handling and single-value rainfall. Passing tests provide evidence for the tested inputs, not a proof for every possible input.